如何编写有利于编译器优化的代码？

原创 IAR Systems 嵌入式系统专家之声 2026年3月18日 08:00 北京

点击上方"蓝字" 关注我们吧！

嵌入式系统专家之声
嵌入式系统专家之声依托嵌入式系统联谊会专家团队，汇集嵌入式系统产、学、研和媒体...
263篇原创内容

公众号

作者：IAR Systems

引言

对于嵌入式系统，最终代码的体积和效率取决于由编译器生成的可执行代码，而非开发人员编写的源代码；但是，源代码的优化可以帮助编译器生成更加优质的可执行代码。因此，开发人员不仅要从整体效率等因素上去构思源代码体系，也要高度关注编译器的性能和编译优化的便捷性。

有优化功能的编译器可生成既小又快的可执行代码，编译器是通过对源代码的重复转换来实现优化。通常，编译器优化会遵循完善的数学或逻辑理论基础。但是某些编译优化则是通过启发式的方法，经验表明，一些代码转换往往会产生更好的代码，或者开拓出进一步编译优化的空间。

编译优化只有少数情况依赖于编译器的黑科技，大多时候编写源代码的方式决定了程序是否可以被编译器优化。在某些情况下，即使对源代码做微小改动，也会对编译器生成的代码效率产生重大影响。本文将讲述在编写代码时需要注意的事项。

循环

下面看似简单的循环会报错吗？

```
1 for(i=0;i!=n; ++i){
2 a[i]=b[i];
3 }
```

虽然不会报错，但其中有几点会影响到编译器生成的代码效率。例如，索引变量的类型应与指针相匹配。

像a[i]这样的数组表达式实际上是*(&a[0]+i*sizeof(a[0])，或者通俗地说：将第i个元素的偏移量加到a的第一个元素的指针上。对于指针运算，索引表达式的类型最好与指针所指向的类型一致（_ _far指针除外，因为其指针所指向的类型和索引表达式的类型不同）。如果索引表达式的类型与指针所指向的类型不匹配，那么在把它与指针相加之前，必须将它强制转换为正确的类型。

如果在应用中，堆栈空间资源（堆栈一般放在RAM中）比代码尺寸资源（代码一般放在ROM或者Flash中）更宝贵，则可以为索引变量选择一个更小的类型来减少堆栈空间的使用，但这往往会

牺牲代码尺寸和执行时间（代码尺寸变大，执行时间变长）。不仅如此，这种转换也会妨碍循环代码的优化。

别名

通常，我们不建议使用全局变量。这是因为您可在程序的任何地方修改全局变量，并且程序会因全局变量的值而变化。这就会形成复杂的依赖关系，使人很难理解程序，也很难确定改变全局变量的值会对程序产生怎样的影响。从优化器的角度来看，这种情况更糟糕，因为通过指针的存储就可以改变任意全局变量的值。如果能通过多种方式访问一个变量，这种情况就会被称为别名，而别名使代码更难优化。

```
char * buf
void clear_buf(){
int i;
For(i=0;i<128;++i){
buf[i]=0;
}
}
```

尽管程序员知道向buf所指向的缓存区进行写操作不会改变这个buf变量本身，但编译器还是不得不做最坏的打算，在循环的每一次迭代中从内存中重新加载buf。

如果将缓存区的地址作为参数传递，而不是使用全局变量，则可以消除别名：

```
void clear_buf(char* buf){
int i;
for(i=0;i<128;++i){
buf[i]=0;
}
}
```

使用这个解决方案后，指针buf就不会被通过指针的存储影响。如此一来，指针buf在循环中就可以保持不变，其值只需在循环前加载一次即可，而不是在每次迭代时都要重新加载。然而，如果需要在不共享调用者 / 被调用者关系的代码段之间传递信息，则直接使用全局变量即可。但是，对于计算密集型任务，尤其是涉及指针操作时，最好使用自动变量。

尽量不用后增量和后减量

在下文中，关于后增量的所有内容也适用于后减量。C语言中关于后增量语义的标准文本指出：后缀＋＋运算符的结果是操作数的值，在得到结果后，操作数的值会递增。虽然微控制器普遍拥有可在加载或存储操作后增加指针的寻址模式，但其中只有很少能以同样的效率处理其他类型的后增量。为符合标准，编译器必须在执行增量之前将操作数复制到一个临时变量。对于直线代码来说，可以从表达式中取出增量，然后放在表达式之后。比如以下表达式：

foo=a[i++];

可以改为：

foo=a[i];

i=i+1;

但如果后增量属于while循环中的条件，又会发生什么？由于在条件后面没有可以插入增量的地方，因此必须在测试前添加增量。对于这些常见但是又与生成可执行代码效率密切相关的设计，诸如IAR Systems的Embedded Workbench这样的工具都在总结了大量实践后提供了优化方案。

比如以下循环：

```
i=0;
while(a[i++]!=0){
……
}
```

应改为

```
loop:
temp=i;            / *保存操作数的值* /
i=temp+1;          / *递增操作数* /
if(a[temp]==0)     / *使用保存的值* /
goto no_loop;
……
goto loop;
no_loop:
```

或

```
loop:
temp=a[i];      / *使用操作数的值* /
i=i+1;          / *递增操作数 * /
if(temp==0)
goto no_loop;
……
goto loop;
no_loop:
```

如果循环后i的值不相关，最好将增量放在循环内，比如以下几乎相同的循环：

```
i=0;
while(a[i]!=0){
++i:
……
}
```

可以在没有临时变量的情况下执行：

```
loop:
```

```
if(a[i]==0)
goto no_loop;
i=i+1;
……
goto loop;
no_loop:
```

优化编译器的开发者们很清楚后增量会使代码编写变得更复杂，尽管我们已尽力去识别这些模式，并尽量消除临时变量，但总有一些情况使我们无法产生有效代码，尤其是遇到比上述更复杂的循环条件时。通常，我们会将一个复杂的表达式分割成若干个更简单的表达式，就像上面的循环条件被分割成一个测试和一个增量那样。

在C++环境中，选择前增量还是后增量的重要性更高。这是因为operator++和operator--都可以以前缀和后缀的形式重载。将运算符作为类对象重载时，虽然没必要模仿基本类型运算符的行为，但也应尽量接近。因此，对于那些可以直观地对对象进行递增和递减的类，例如迭代器，通常会有前缀（operator++()和operator--()）和后缀形式（operator++(int)和operator--(int)）。

为了模拟基本类型的前缀++的行为，operator++()可以修改对象并返回对修改后对象的引用。那么模拟基本类型的后缀++的行为会怎样？您还记得吗？后缀++运算符的结果是操作数的值，在得到结果后，操作数的值会递增。就像上面的非直线代码一样，operator++(int)的实现者必须复制原始对象，修改原始对象，并按值返回副本。由于存在复制操作，因此operator++(int)的开销要高于operator++()。

对于基本类型，如果忽略i++的结果，优化器通常可以消除不必要的复制，但优化器不能将对一个重载运算符的调用变为另一个。如果出于习惯编写i++而不是++i，就会调用开销更大的增量运算符。

结语

当然，软件开发工作并不是只要求开发人员去“将就”编译器，他们与编译器之间的相互协同是快速而高效地完成编程工作的基础之一。此外，从编译器的发展过程来看，它们不仅要跟随技术和语言的演进而迭代和创新，而且还要广泛参考更多的开发习惯，那些历史更悠久、使用更广泛的编译器可以为开发人员带来更高的效率。

因此，在了解了如何编写利于一款优秀编译器优化的代码之后，用户们的工作效率就可以事半功倍。本文中提到的这些原理和技巧，也是IAR Systems这样的公司长时间总结的最优实践，而且都可以在该公司的Embedded Workbench中进行验证和探索，在其工具界面中可以查看代码的执行时间和代码尺寸，从而找到最佳解决方案。

（本文由《单片机与嵌入式系统应用》杂志社授权发表，原文刊发在2022年第2期）

END

嵌入式系统专家之声推荐搜索

嵌入式系统 | RTOS | 开源

【点击上方👆搜索词条可查看号内更多其他内容】



关注我们，了解更多精彩内容

转发，点赞，在看，安排一下?